Performance Evaluation of FlashAttention-2 on CPU

Yu-Ting Lee
Graduate Institute of Communication Engineering
National Taiwan University, Taipei, Taiwan
r14942088@ntu.edu.tw

***Abstract*—FlashAttention has revolutionized transformer training and inference on GPU by minimizing memory I/O, yet its behavior on CPU remains underexplored. This report provides a performance evaluation of FlashAttention-2 for inference on a single CPU. Specifically, we evaluate the FlashAttention-2 forward pass across various across various single- and multi-threaded implementations in C++ and Python, comparing them against standard attention baselines. We also include a line-by-line complexity analysis and profile algorithmic runtime distributions across varying sequence lengths and cache budgets. Our results demonstrate that while standard attention implemented in C++ remains optimal for short sequences, parallelized FlashAttention-2 forward pass yields up to a 7.9× speedup on long sequences. On the other hand, we find that suboptimal tile sizes whether too small or too large can lead to performance inferior to the baseline. Our analysis serves as a preliminary exploration of efficient attention computations in resource-constrained settings, including CPU-only inference and edge deployments.**

Code is available at:
https://github.com/Yu-TingLee/flashattn2-cpu.

I. Introduction

The attention mechanism serves as the fundamental building block of transformer models [1], driving their success across various tasks. However, the standard implementation of attention incurs quadratic computational and I/O complexity with respect to sequence length. As the sequence length grows, the intermediate results of attention often exceed the capacity of fast on-chip memory (SRAM or cache), forcing the system to rely on off-chip memory (HBM). This creates a significant performance bottleneck due to I/O costs.

To address this I/O bottleneck, FlashAttention-2 [2] was introduced as a hardware-aware, exact attention algorithm. By utilizing tiling and kernel fusion, it computes the attention output in a single pass without writing large intermediate matrices. While FlashAttention-2 has been extensively optimized and adopted for GPU, its performance on general-purpose CPU remains largely unexplored. This gap is significant because CPU inference remains critical for resource-constrained environments [3], such as edge computing, where high-end GPUs are often unavailable.

In this report, we investigate the efficacy of FlashAttention-2 for inference for inference on a single CPU. Unlike GPU, which leverages the SIMT execution model for massive parallel throughput and high bandwidth memory, CPU relies on a smaller number of cores with cache hierarchies (L1, L2, L3) and achieves parallelism through SIMD vector units and multi-threading. To evaluate whether these I/O-efficient design principles transfer effectively to this hardware paradigm, we benchmark several single- and multi-threaded implementations of the FlashAttention-2 forward pass in C++ and Python. We further include a line-by-line complexity analysis and conduct profiling experiments on the runtime distributions of algorithmic steps.

The rest of the report is organized as follows. Section II provides background on efficient attention computations and the architectural differences between CPU and GPU. Section III describes the standard implementation of attention as well as its I/O and computational complexity. Section IV provides derivations of the two-pass tiled attention and the one-pass online attention, followed by the details of the FlashAttention-2 forward pass and its complexity analysis. Experimental settings and results are presented in Sections V and VI, respectively. We discuss limitations in Section VII and conclude this report in Section VIII.

II. Background

A. Efficient Attention Computations

To compute attention efficiently on long contexts, numerous methods have been proposed to approximate attention [4]–[10]. These encompass sparse attention patterns [4], [8]–[10], low-rank approximations [6], [7], and hybrid approaches [5]. More recently, IceFormer [11] and NoMAD-attention [12] accelerate attention inference on CPU via approximate methods, using k-nearest-neighbor search and SIMD in-register lookups, respectively. However, these approximation methods typically trade model accuracy for theoretical efficiency and may fail to achieve true wall-clock speedups against standard attention.

In contrast, exact attention methods compute the mathematically equivalent attention matrix while optimizing data movement. Specifically, FlashAttention [13] and FlashAttention-2 [2] demonstrated that attention is bottlenecked by I/O cost, achieving substantial speedups on GPU by minimizing memory accesses. However, whether these hardware-aware advantages translate to CPU remains largely underexplored.

B. Hardware Characteristics: CPU vs. GPU

1) Performance Characteristics: GPU consists of compute elements (e.g., floating-point arithmetic units) and a memory hierarchy. The memory hierarchy comprises high bandwidth memory (HBM) and on-chip SRAM. As an example, the A100 GPU has 40-80 GB of HBM with bandwidth 1.5-2.0 TB/s and 192 KB of on-chip SRAM per each of 108 streaming multiprocessors with bandwidth estimated around

19 TB/s. CPU relies on a multi-level cache hierarchy (L1, L2, L3) backed by off-chip DRAM, where caches are implicitly managed by hardware. As a reference, the i9-11900K used in our experiments has 512 KiB of L2 cache per core and 16 MiB of shared L3 cache, with a peak DRAM bandwidth of approximately 38.4 GB/s.

2) Execution Model: GPU achieves massive parallelism via the SIMT execution model, where threads execute in groups of 32 (warps) across streaming multiprocessors. Modern GPU also features specialized matrix multiply units (e.g., Tensor Cores on Nvidia GPUs) that can deliver over an order of magnitude higher throughput than general-purpose floating-point units. For instance, the A100 achieves 312 TFLOPs/s for FP16/BF16 matmul versus 19.5 TFLOPs/s for non-matmul FP32. CPU relies on a smaller number of cores optimized for complex instruction streams, with parallelism expressed through both SIMD vector units and thread-level concurrency.

III. An Analysis of Naive Attention

Let $W_Q \in \mathbb{R}^{d\times d}, W_K \in \mathbb{R}^{d\times d}, W_V \in \mathbb{R}^{d\times d}$ be the trainable weights and $\widetilde{Z} \in \mathbb{R}^{T\times d}$ be the input matrix. For a transformer model, T specifies the input sequence length and d is the head dimension. For convenience, we define $\mathbf{Q} := \widetilde{Z}W_Q \in \mathbb{R}^{T\times d}, \mathbf{K} := \widetilde{Z}W_K \in \mathbb{R}^{T\times d}, \mathbf{V} := \widetilde{Z}W_V \in \mathbb{R}^{T\times d}$. The computation of attention can be summarized as

$$\mathrm{softmax}\left(\frac{\widetilde{Z}W_Q W_K^\top \widetilde{Z}^\top}{\sqrt{d}}\right)\widetilde{Z}W_V, \tag{1}$$

where the softmax is applied row-wise. I.e., it transforms each row $\mathbf{z}$ of an input matrix in the following way:

$$\mathrm{softmax}(\mathbf{z}) := \left[\frac{\exp(z_1)}{\sum_j \exp(z_j)}, \frac{\exp(z_2)}{\sum_j \exp(z_j)}, \ldots, \frac{\exp(z_T)}{\sum_j \exp(z_j)}\right]. \tag{2}$$

We write the output matrix of the attention as

$$\mathbf{O} := \mathrm{softmax}(\mathbf{QK}^\top/\sqrt{d})\mathbf{V} \in \mathbb{R}^{T\times d}. \tag{3}$$

Fig. 1 presents a visualization of the computation of attention. In this work, we refer to this standard, unoptimized computation of attention as *naive attention*.

For simplicity, we assume a simplified two-level memory hierarchy only consisting of on-chip SRAM and off-chip memory. In general, the input sequence length is much larger than the head dimension, i.e., $T \gg d$, and we may assume the intermediate $T \times T$ matrices $\mathbf{QK}^\top$ and $\mathrm{softmax}(\mathbf{QK}^\top/\sqrt{d})$ cannot be stored in on-chip SRAM.

A. I/O and Computational Complexity of Naive Attention

We are ready to analyze the I/O and computational complexity of naive attention.

1) I/O Complexity: We consider the computations of $\mathbf{QK}^\top \in \mathbb{R}^{T\times T}$, $\mathrm{softmax}(\mathbf{QK}^\top/\sqrt{d}) \in \mathbb{R}^{T\times T}$, and $\mathrm{softmax}(\mathbf{QK}^\top/\sqrt{d})\mathbf{V} \in \mathbb{R}^{T\times d}$ and the corresponding major memory accesses. To compute attention, we must sequentially:

- Write T^2 elements to off-chip memory to store $\mathbf{QK}^\top$.
- Read T^2 elements by loading $\mathbf{QK}^\top$ from off-chip memory to calculate $\mathrm{softmax}(\mathbf{QK}^\top/\sqrt{d})$, and write T^2 elements back to off-chip memory.

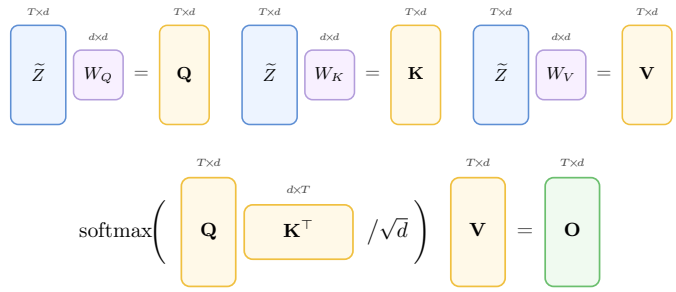

Fig. 1: **Computation of standard attention.** Input matrix $\widetilde{Z}$ is projected by trainable weights W_Q, W_K, and W_V to obtain the query matrix $\mathbf{Q}$, key matrix $\mathbf{K}$, and value matrix $\mathbf{V}$. Output matrix $\mathbf{O}$ is obtained by applying row-wise softmax to the scaled product $\mathbf{QK}^\top/\sqrt{d}$, then multiplying by $\mathbf{V}$.

- Read T^2 elements by loading $\mathrm{softmax}(\mathbf{QK}^\top/\sqrt{d})$ from off-chip memory to calculate $\mathrm{softmax}(\mathbf{QK}^\top/\sqrt{d})\mathbf{V}$.

We conclude that naive attention induces $4T^2 = \mathcal{O}(T^2)$ memory accesses.

2) Computational Complexity: On the other hand, the computational complexity for each operation is, respectively,

- $\mathbf{QK}^\top$: $2(TdT) = 2dT^2$,
- $\mathrm{softmax}(\mathbf{QK}^\top/\sqrt{d})$: $4T^2$ (each row takes $4T$ operations for exponentiation, summation, and normalization),
- $\mathrm{softmax}(\mathbf{QK}^\top/\sqrt{d})\mathbf{V}$: $2(TdT) = 2dT^2$.

Summing up, naive attention induces $(4dT^2 + 4T^2) = \mathcal{O}(T^2 d)$ operations.

As we can see, naive attention induces memory accesses that are quadratic in T, making it less efficient for long-sequence workloads. It is thus crucial to design I/O-efficient attention algorithms that reduce memory accesses while preserving computational equivalence to naive attention. Specifically, it is desirable that the high speed of on-chip SRAM can be fully utilized.

IV. An I/O-Efficient Attention Algorithm: FlashAttention-2

In this section, we motivate and develop the FlashAttention-2 algorithm [2], detailing its core concepts and presenting the forward pass in Algorithm 2. Assume the on-chip SRAM size M satisfies $d \le M \le Td$. Since M is insufficient to store any $T\times T$ matrices, naive attention must materialize and reload such matrices, leading to $\mathcal{O}(T^2)$ memory accesses. FlashAttention-2 mitigates this bottleneck by partitioning $\mathbf{Q}$, $\mathbf{K}$ and $\mathbf{V}$ into row blocks $\mathbf{Q}_{I_1}, \ldots, \mathbf{Q}_{I_{T_r}}$, $\mathbf{K}_{J_1}, \ldots, \mathbf{K}_{J_{T_c}}$, and $\mathbf{V}_{J_1}, \ldots, \mathbf{V}_{J_{T_c}}$ that fit entirely with on-chip SRAM. The output matrix $\mathbf{O}$ is similarly partitioned into row blocks $\mathbf{O}_{I_1}, \ldots, \mathbf{O}_{I_{T_r}}$. This technique is commonly referred to as *tiling* in literature.

However, tiling alone does not resolve the memory bottleneck inherent to softmax and attention computation. First, the softmax for a given row depends on the sum of exponentials across the entire row, not just a single block. Therefore, a standard implementation requires two passes over the input: (1) a pass to compute the sum of exponentials, and (2) a pass

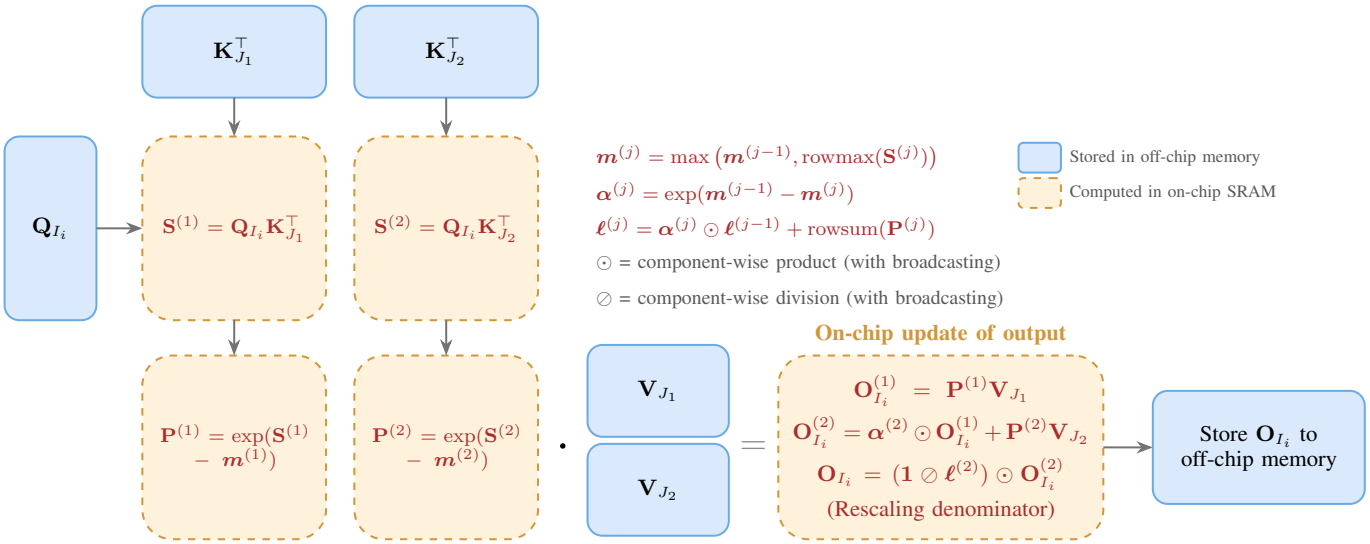

Fig. 2: **Computation of the FlashAttention-2 forward pass (Algorithm 2).** We assume only $T_c = 2$ column blocks. For a fixed row block I_i, the key $\mathbf{K}$ and value $\mathbf{V}$ blocks are loaded sequentially into on-chip SRAM, where the intermediate matrices $\mathbf{S}^{(j)}$, $\mathbf{P}^{(j)}$, and the running statistics $\boldsymbol{m}^{(j)}$, $\boldsymbol{\ell}^{(j)}$ are computed and used to update $\mathbf{O}_{I_i}$ in place. After all column blocks are processed, the output is rescaled by $\boldsymbol{\ell}$ and stored to off-chip memory. This avoids materializing the $T \times T$ intermediate matrices $\mathbf{S}$ and $\mathbf{P}$ in off-chip memory.

to compute the normalized output. For numerical stability, it is also standard practice to subtract the maximum element from each row before exponentiation. This trick is known as the *safe softmax*. Consequently, a safe softmax naively requires three passes: (1) a pass to compute the row-wise maximum, (2) a pass to compute the sum of exponentials, and (3) a pass to compute the normalized output. For long sequences, these redundant memory accesses degrade performance.

Second, when computing tiled attention, the non-linearity of the softmax function prevents us from naively summing partial results to update the output matrix $\mathbf{O}$. Instead, we must rescale the existing partial results to account for the new normalization factor before adding the contribution from the current block of $\mathbf{K}$ and $\mathbf{V}$.

For efficient computation, FlashAttention-2 utilizes *kernel fusion* to achieve a *one-pass online attention*, which iteratively updates the output block $\mathbf{O}_{I_i}$ as new row-blocks of $\mathbf{K}_{J_j}^\top$ and $\mathbf{V}_{J_j}$ are processed. Although *kernel fusion* and *one-pass online attention* may sound fancy and possibly intimidating at first, it's quite simple mathematically, as we will explain below.

A. One-Pass Online Attention

One-pass online attention is the functional core of FlashAttention-2. We first establish the mathematical foundation before analyzing the algorithm's memory and computational complexity.

1) Safe Softmax: We first review safe softmax, which is a numerically stable way to compute softmax. Recall that softmax (equation (2)) exponentializes each element of a vector $\mathbf{z}$ and normalizes them by the sum of exponentials. This creates numerical instability when some elements of $\mathbf{z}$ are large, leading to overflow during exponentiation. For example, float32 can represent numbers up to approximately 3.4×10^{38}, which means $\exp(89) \approx 8.6 \times 10^{38}$ already overflows. To mitigate this, safe softmax simply subtracts the maximum element from $\mathbf{z}$ before exponentiation:

$$\frac{\exp(z_i)}{\sum_j \exp(z_j)} = \frac{\exp(z_i - \max_k z_k)}{\sum_j \exp(z_j - \max_k z_k)}. \tag{4}$$

Note that the equality holds trivially as $\exp(z_i - \max_k z_k) = \exp(z_i)/\exp(\max_k z_k)$. We are ready to introduce the two-pass tiled attention, which is also mathematically straightforward.

2) Two-Pass Tiled Attention: First, we partition $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ into the following row blocks:

$$\mathbf{Q} = \begin{bmatrix} Q_{I_1} \\ Q_{I_2} \\ \vdots \\ Q_{I_{T_r}} \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} K_{J_1} \\ K_{J_2} \\ \vdots \\ K_{J_{T_c}} \end{bmatrix}, \quad \mathbf{V} = \begin{bmatrix} V_{J_1} \\ V_{J_2} \\ \vdots \\ V_{J_{T_c}} \end{bmatrix}$$

where $|I_i| = T/T_r =: |I|$ for all $i = 1, 2, \ldots, T_r$ and $|J_j| = T/T_c =: |J|$ for all $j = 1, 2, \ldots, T_c$. For each block I_i, we may rewrite attention as

$$\mathrm{softmax}\left(\frac{\mathbf{Q}_{I_i}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V} = \mathrm{softmax}\left(\frac{1}{\sqrt{d}}\left[\mathbf{Q}_{I_i}\mathbf{K}_{J_1}^\top, \ldots, \mathbf{Q}_{I_i}\mathbf{K}_{J_{T_c}}^\top\right]\right)\begin{bmatrix} \mathbf{V}_{J_1} \\ \vdots \\ \mathbf{V}_{J_{T_c}} \end{bmatrix}. \tag{5}$$

Let $\mathbf{z}$ be a row of $\mathbf{Q}_{I_i}\mathbf{K}^\top$. Note that for $j = 1, 2, \ldots, T_c$, the following recurrence holds:

$$m_j = \max(m_{j-1}, \max_{t \in J_j} z_t) \tag{6}$$

$$d_j = d_{j-1} \cdot \exp(m_{j-1} - m_j) + \sum_{t \in J_j} \exp(z_t - m_j) \tag{7}$$

where $m_j = \max_{t \in J_1 \cup \cdots \cup J_j} z_t$ and $d_j = \sum_{t \in J_1 \cup \cdots \cup J_j} \exp(z_t - m_j)$. That is, we can actually compute the maximum and sum of exponentials iteratively over J_j. Using the above recurrence and simple matrix algebra, the tiled attention can be computed in two passes (Algorithm 1):

Algorithm 1 Two-Pass Tiled Attention

Divide $\mathbf{Q}$ into row-block matrices $\mathbf{Q}_{I_1}, \ldots \mathbf{Q}_{I_{T_r}}$ of size $|I| \times d$ and divide $\mathbf{K}, \mathbf{V}$ into $\mathbf{K}_{J_1}, \ldots, \mathbf{K}_{J_{T_c}}$ and $\mathbf{V}_{J_1}, \ldots, \mathbf{V}_{J_{T_c}}$ of size $|J| \times d$ each.
Divide the output $\mathbf{O}$ into row-block matrices $\mathbf{O}_{I_1}, \ldots \mathbf{O}_{I_{T_r}}$ of size $|I| \times d$ each.
for $i = 1$ to T_r **do**
  Load $\mathbf{Q}_{I_i}$ to on-chip SRAM.
  Initialize $\mathbf{O}_{I_i} \leftarrow \mathbf{0}_{|I_i| \times d}$, $\boldsymbol{\ell} \leftarrow \mathbf{0}_{|I_i|}$ and $\boldsymbol{m} \leftarrow (-\infty)_{|I_i|}$ on chip.
  for $j = 1$ to T_c **do**
    Load $\mathbf{K}_{J_j}$ to on-chip SRAM.
    Compute $\mathbf{S}_j \leftarrow \mathbf{Q}_{I_i}\mathbf{K}_{J_j}^\top$ on chip.
    Compute $\boldsymbol{m}_j \leftarrow \max(\boldsymbol{m}_{j-1}, \mathrm{rowmax}(\mathbf{S}_j))$ on chip.
    # $\odot$ = component-wise product
    Compute $\boldsymbol{\ell}_j \leftarrow \exp(\boldsymbol{m}_{j-1} - \boldsymbol{m}_j) \odot \boldsymbol{\ell}_{j-1} + \mathrm{rowsum}(\exp(\mathbf{S}_j - \boldsymbol{m}_j))$ on chip.
    Store $\mathbf{S}_j$ to off-chip memory.
  end for
  for $j = 1$ to T_c **do**
    Load $\mathbf{S}_j$ and $\mathbf{V}_{J_j}$ to on-chip SRAM.
    # $\oslash$ = component-wise division; $\odot$ = component-wise product
    Compute $\mathbf{P}_j \leftarrow (\mathbf{1} \oslash \boldsymbol{\ell}_{T_c}) \odot \exp(\mathbf{S}_j - \boldsymbol{m}_{T_c})$ on chip.
    Compute $\mathbf{O}_{I_i} \leftarrow \mathbf{O}_{I_i} + \mathbf{P}_j\mathbf{V}_{J_j}$ on chip.
  end for
  Store $\mathbf{O}_{I_i}$ to off-chip memory.
end for

3) One-Pass Online Attention: While the two-pass approach is an improvement, we can further fuse the passes into a single pass. Let $\mathbf{O}_{I_i}^{(j)} = \sum_{k=1}^{j} \exp(\mathbf{S}_k - \boldsymbol{m}_j)\mathbf{V}_{J_k}$. We observe the following relationship:

$$\begin{aligned}
\mathbf{O}_{I_i}^{(j)} &= \left[\sum_{k=1}^{j-1} \exp(\mathbf{S}_k - \boldsymbol{m}_j)\mathbf{V}_{J_k}\right] + \exp(\mathbf{S}_j - \boldsymbol{m}_j)\mathbf{V}_{J_j} \\
&= \exp(\boldsymbol{m}_{j-1} - \boldsymbol{m}_j) \odot \left[\sum_{k=1}^{j-1} \exp(\mathbf{S}_k - \boldsymbol{m}_{j-1})\mathbf{V}_{J_k}\right] \\
&\quad + \exp(\mathbf{S}_j - \boldsymbol{m}_j)\mathbf{V}_{J_j} \\
&= \exp(\boldsymbol{m}_{j-1} - \boldsymbol{m}_j) \odot \mathbf{O}_{I_i}^{(j-1)} + \exp(\mathbf{S}_j - \boldsymbol{m}_j)\mathbf{V}_{J_j}
\end{aligned} \tag{8}$$

Now, one sees that

$$\mathbf{O}_{I_i} = (\mathbf{1} \oslash \boldsymbol{\ell}_{T_c}) \odot \mathbf{O}_{I_i}^{(T_c)}. \tag{9}$$

With some algebraic manipulations, we have successfully removed the forward dependency on $\boldsymbol{\ell}_{T_c}$ and $\boldsymbol{m}_{T_c}$. Now, we can fuse the two loops into one and arrive at the FlashAttention-2 forward pass (Algorithm 2). We provide a visual illustration in Fig. 2.

Algorithm 2 FlashAttention-2 Forward Pass

Divide $\mathbf{Q}$ into row-block matrices $\mathbf{Q}_{I_1}, \ldots \mathbf{Q}_{I_{T_r}}$ of size $|I| \times d$ and divide $\mathbf{K}, \mathbf{V}$ into $\mathbf{K}_{J_1}, \ldots, \mathbf{K}_{J_{T_c}}$ and $\mathbf{V}_{J_1}, \ldots, \mathbf{V}_{J_{T_c}}$ of size $|J| \times d$ each.
Divide the output $\mathbf{O}$ into row-block matrices $\mathbf{O}_{I_1}, \ldots \mathbf{O}_{I_{T_r}}$ of size $|I| \times d$ each.
for $i = 1$ to T_r **do**
  Load $\mathbf{Q}_{I_i}$ to on-chip SRAM.
  Initialize $\mathbf{O}_{I_i} \leftarrow \mathbf{0}_{|I_i| \times d}$, $\boldsymbol{\ell} \leftarrow \mathbf{0}_{|I_i|}$ and $\boldsymbol{m} \leftarrow (-\infty)_{|I_i|}$ on chip.
  for $j = 1$ to T_c **do**
    Load $\mathbf{K}_{J_j}$ and $\mathbf{V}_{J_j}$ to on-chip SRAM.
    Compute $\mathbf{S} \leftarrow \mathbf{Q}_{I_i}\mathbf{K}_{J_j}^\top$ on chip.
    # $\oslash$ = component-wise division; $\odot$ = component-wise product
    Compute $\boldsymbol{m}_{\mathrm{new}} \leftarrow \max(\boldsymbol{m}, \mathrm{rowmax}(\mathbf{S}))$, $\mathbf{P} \leftarrow \exp(\mathbf{S} - \boldsymbol{m}_{\mathrm{new}})$, $\boldsymbol{\ell}_{\mathrm{new}} \leftarrow \boldsymbol{\alpha} \odot \boldsymbol{\ell} + \mathrm{rowsum}(\mathbf{P})$ on chip, where $\boldsymbol{\alpha} = \exp(\boldsymbol{m} - \boldsymbol{m}_{\mathrm{new}})$.
    Compute $\mathbf{O}_{I_i} \leftarrow \boldsymbol{\alpha} \odot \mathbf{O}_{I_i} + \mathbf{P}\mathbf{V}_{J_j}$ on chip.
    Compute $\boldsymbol{m} \leftarrow \boldsymbol{m}_{\mathrm{new}}$, $\boldsymbol{\ell} \leftarrow \boldsymbol{\ell}_{\mathrm{new}}$ on chip.
  end for
  Compute $\mathbf{O}_{I_i} \leftarrow (\mathbf{1} \oslash \boldsymbol{\ell}) \odot \mathbf{O}_{I_i}$ on chip.
  Store $\mathbf{O}_{I_i}$ to off-chip memory.
end for

B. I/O and Computational Complexity Analysis of FlashAttention-2

We analyze the I/O and computational complexity of the FlashAttention-2 forward pass.

1) I/O Complexity: For each outer iteration i:

- We load $\mathbf{Q}_{I_i}$ and initialize $\mathbf{O}_{I_i}$ once. This costs $\mathcal{O}(|I|d)$.
- We iterate through the inner loop $j = 1, \ldots, T_c$ and load $\mathbf{K}_{J_j}$ and $\mathbf{V}_{J_j}$ for T_c times. This costs $\mathcal{O}((|J| \times T_c)d) = \mathcal{O}(Td)$.

- Finally, we write the result $\mathbf{O}_{I_i}$ back to off-chip memory. This costs $\mathcal{O}(|I|d)$.

Summing over the T_r outer iterations, the number of memory accesses is $\mathcal{O}(T_r \times Td) = \mathcal{O}(\frac{T^2 d}{|I|})$. Choosing $|I| = \Theta(M/d)$, we have total memory accesses $\mathcal{O}(\frac{T^2 d^2}{M})$. For typical values of T and d, this is much more I/O efficient.

2) Computational Complexity: For each (I_i, J_j), the matrix products $\mathbf{Q}_{I_i}\mathbf{K}_{J_j}^\top$ and $\mathbf{P}\mathbf{V}_{J_j}$ each costs $\mathcal{O}(|I||J|d)$. Summing over $T_r T_c = \frac{T}{|I|}\frac{T}{|J|}$ blocks, the computational complexity is $\mathcal{O}(T^2 d)$, which is the same as naive attention.

While the above analysis confirms that FlashAttention-2 achieves better memory complexity than naive attention, we also provide a finer-grained, line-by-line theoretical analysis to contextualize our subsequent empirical runtime breakdowns.

C. Line-by-Line Complexity Analysis of FlashAttention-2

1) Line-by-Line I/O Complexity: We calculate the exact number of elements transferred between SRAM and off-chip memory.

- **Line 4** (Load $\mathbf{Q}_{I_i}$ to on chip SRAM). Each block $\mathbf{Q}_{I_i}$ contains $|I|d$ elements. So, line 4 yields Td memory access cost.
- **Line 7** (Load $\mathbf{K}_{J_j}$ and $\mathbf{V}_{J_j}$ to on chip SRAM). Each (i, j) iteration reads $2|J|d$ elements. So, line 7 yields $2\frac{T^2 d}{|I|}$ memory access cost.
- **Line 15** (Store $\mathbf{O}_{I_i}$ to off-chip memory). Each block $\mathbf{O}_{I_i}$ contains $|I|d$ elements. So, line 15 yields Td memory access cost.

2) Line-by-Line Computational Complexity: Similarly, we break down the exact operation counts for matrix multiplication and online softmax.

- **Line 8** (Compute $\mathbf{S} \leftarrow \mathbf{Q}_{I_i}\mathbf{K}_{J_j}^\top$ on chip). Each (i, j) iteration costs $2|I||J|d$ operations. So, line 8 yields $2dT^2$ total computational cost.
- **Line 10** (Compute $\boldsymbol{m}_{\text{new}}$, $\mathbf{P}$ and $\boldsymbol{\ell}_{\text{new}}$ on chip). Note that $\text{rowmax}(\mathbf{S})$ requires $|I|(|J| - 1)$ operations. Updating $\boldsymbol{m}_{\text{new}}$ requires $|I|$ comparisons. Updating $\mathbf{P}$ requires $2|I||J|$ operations. Updating $\boldsymbol{\alpha} \odot \boldsymbol{\ell}$ requires $3|I|$ operations. Updating $\text{rowsum}(\mathbf{P})$ requires $|I|(|J| - 1)$ operations. Updating $\boldsymbol{\ell}_{\text{new}}$ requires additional $|I|$ additions. Summing up, line 10 requires $4|I||J| + 3|I|$ operations per execution. So, line 10 yields $4T^2 + 3\frac{T^2}{|J|}$ total computational cost.
- **Line 11** (Compute $\mathbf{O}_{I_i} \leftarrow \boldsymbol{\alpha} \odot \mathbf{O}_{I_i} + \mathbf{P}\mathbf{V}_{J_j}$ on chip). Updating $\boldsymbol{\alpha}$ requires $2|I|$ operations. Component-wise product with $\mathbf{O}_{I_i}$ requires $|I|d$ operations; $\mathbf{P}\mathbf{V}_{J_j}$ requires $2|I||J|d$ operations. Updating $\mathbf{O}_{I_i,;}$ requires additional $|I|d$ additions. Summing up, line 11 requires $2|I|(1 + d + |J|d)$ operations per execution. So, line 11 yields $2T^2 d + 2T^2\frac{d}{|J|} + 2\frac{T^2}{|J|}$ total computational cost.
- **Line 14** (Compute $\mathbf{O}_{I_i} \leftarrow (\mathbf{1} \oslash \boldsymbol{\ell}) \odot \mathbf{O}_{I_i}$ on chip). Note that $\mathbf{1} \oslash \boldsymbol{\ell}$ requires $|I|$ operations. Component-wise product with $\mathbf{O}_{I_i}$ requires $|I|d$ operations. Summing up, line 14 requires $|I| + |I|d$ operations per execution. So, line 14 yields $Td + T$ total computational cost.

Based on the above analysis, we note that Line 7 dominates the I/O cost, Line 8 and Line 11 dominate the computation complexity, all by the scale of $T^2 d$. We investigate the empirical runtime distribution of these major algorithmic steps (Lines 7, 8, 10, and 11) in Section VI-B.

V. Experimental Settings

TABLE I: CPU, compiler, and dependencies.

CPU Info	
Model Name	11th Gen Intel(R) Core(TM) i9-11900K @ 3.50GHz
Vendor	GenuineIntel
CPU Family / Model / Stepping	6 / 167 / 1
Cores / Threads	8 Physical / 16 Logical
L2 Cache Size	512 KiB per core
L3 Cache Size	16 MiB (Shared)
Address Sizes	39 bits physical, 48 bits virtual
Compiler and Dependencies	
CXX	g++
CXXFLAGS	`-std=c++17 -O3 -march=native -ffast-math`
C++ Dependencies	Eigen3, OpenMP
Python Dependencies	NumPy, Numba

We first develop a Python script to generate 50 reference test sets containing inputs $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ and their corresponding ground-truth outputs $\mathbf{O}$. Using these reference values, we verify the correctness of all implementations within numerical tolerance.

All C++ implementations utilize Eigen3, which is a high-performance library for linear algebra. Naive attention is implemented in both C++ and Python. For FlashAttention-2 (Algorithm 2), we evaluate four variants: C++ single-threaded (ST), C++ multi-threaded (MT), Python (NumPy), and Python (Numba). The C++ implementations are compiled via CMake in Release mode with `-O3`, `-march=native`, and `-ffast-math`. Multi-threaded variants utilize 16 threads with OpenMP. The NumPy implementations use a single Python thread with default OpenBLAS multi-threading for matrix multiplications. The Numba implementation uses parallel range for multi-threading (16 threads) on the other loop and the `@jit` decorator. See Table I for the hardware and software specifications.

For performance evaluation, we measure the runtime of each implementation across sequence lengths $T \in \{256, 512, 1024, 2048, 4096, 8192\}$ and head dimensions $d \in \{64, 128\}$. We manually set the cache budget M to simulate different on-chip SRAM sizes, sweeping values from 32 KiB to 1 MiB. Note that M controls the tiled block size. Specifically, we compute the row block size $|I|$ and column block size $|J|$ as: $|I| = \lfloor M/(16d) \rfloor, |J| = \min(|I|, d)$. For each configuration, we aggregate the results over the 50 reference test sets.

We also conduct profiling experiments on the C++ (ST) and Python (NumPy) implementations to break down the runtime of specific lines in Algorithm 2. For profiling, we sweep sequence lengths $T \in \{512, 2048, 8192\}$, head dimensions $d \in \{64, 128\}$, and select cache budgets $M \in \{16\,\text{KiB}, 256\,\text{KiB}, 1\,\text{MiB}\}$. For each configuration, the profiling results are aggregated over d and the 50 reference test sets. This

analysis accompanies the theoretical line-by-line complexity derived in Section IV-B.

VI. Experimental Results

In this section, we present our experimental results on runtime performance and profiling analysis.

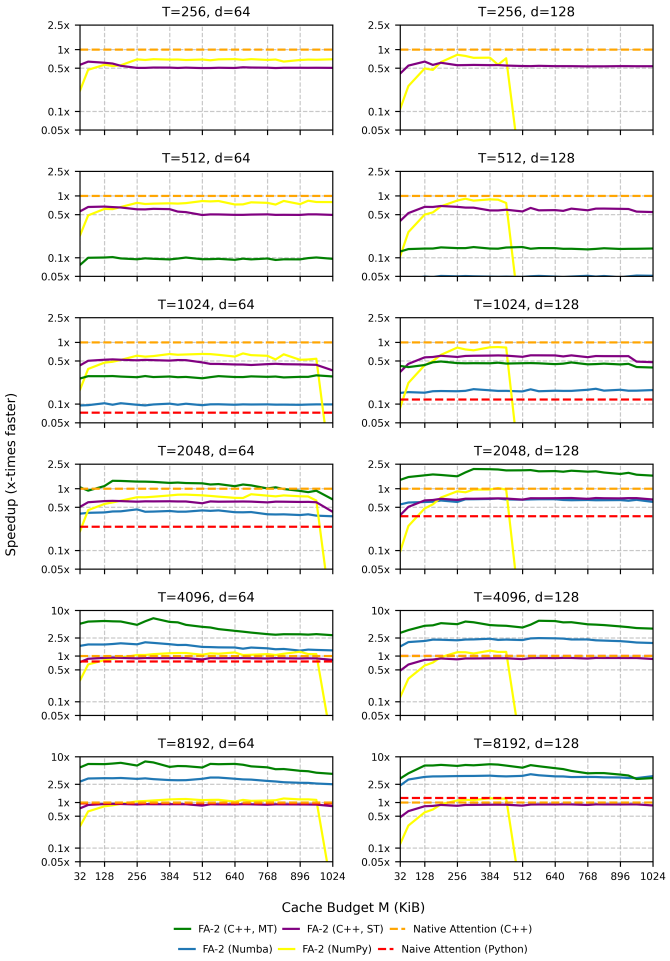

Fig. 3: Speedup compared to Naive Attention (C++) across different sequence lengths T, head dimensions d, and cache budgets M. Results show that Naive Attention (C++) remains the fastest across all configurations for short sequences. FA-2 (C++, MT) consistently outperforms all other implementations when $T \geq 2048$.

A. Runtime Performance across Sequence Lengths, Head Dimensions, and Cache Budgets

Figure 3 presents the relative speedup compared to Naive Attention (C++) across different sequence lengths T, head dimensions d, and cache budgets M. Our results across these configurations reveal distinct performance patterns.

1) Impact of Sequence Length T: The multi-threaded C++ implementation, FA-2 (C++, MT), consistently outperforms all other implementations when $T \geq 2048$. It even achieves a significant speedup of up to 7.9× (an 87% runtime reduction) on long sequences. However, for short sequences ($T \leq 1024$), Naive Attention (C++) remains the fastest across all configurations. This likely occurs because the naive implementation executes as a streamlined, single computation without the overhead of managing tiles.

Interestingly, we also note that single-threaded FA-2 (C++, ST) falls behind Naive Attention (C++) across almost all configurations. This contrast highlights a crucial takeaway: while single-threaded FA-2 incurs overhead that prevents it from surpassing a well-optimized naive C++ implementation, coupling this I/O efficiency with parallelization unlocks substantial performance gains.

2) Impact of Cache Budget M: For all configurations, the curves of FA-2 (Numba), FA-2 (C++, ST), and FA-2 (C++, MT) remain almost flat. This suggests that these implementations are insensitive to cache budget M within the tested range. Conversely, FA-2 (Numpy) exhibits a U-curve performance pattern as M varies. In particular, for $d = 128$, we observe a sharp performance cliff at $M = 512$ KiB. We hypothesize that this U-curve behavior can be explained by three regimes:

- Small M: When blocks are small (e.g., $M \leq 256$ KiB), the loop and tiling overhead degrade the runtime performance.
- Optimal M: Performance peaks at intermediate sizes, where blocks are large enough to reduce loop overhead but not so large as to degrade performance.
- Large M: As blocks get larger (e.g., $M \geq 512$ KiB), we see a sharp performance drop. While the exact reason for this phenomenon requires further investigation, it appears to coincide with L2 cache capacity limits.

Indeed, we can estimate the total working set:

- $M = 512$ KiB: With $|I| = 256$, the blocks $\mathbf{Q}_{I_i}$, $\mathbf{O}_{I_i}$, $\mathbf{S}$, and $\mathbf{P}$ are 128 KiB each. Hence, the total working set size is 640 KiB, which exceeds the 512 KiB L2 cache limit, possibly causing the observed performance degradation.

3) Impact of Head Dimension d: In the FA-2 (NumPy) implementation, we observe that the $d = 64$ configuration sustains performance at much larger block sizes than $d = 128$. A severe performance drop is observed only after the cache budget reaches $M = 960$ KiB; at this point, the estimated working set for the active blocks is 992 KiB, which greatly exceeds the L2 cache. The exact architectural or software-level reasons for this delayed performance cliff remain unknown. In contrast, the C++ and Numba implementations remain more robust and insensitive to the change in head dimension d.

B. Runtime Distribution of Algorithmic Steps

Figure 4 presents the runtime breakdown of the major algorithmic steps. We observe three distinct behaviors dependent on the block size M:

- Loop Overhead and I/O Cost: At the smallest block size ($M = 16$ KiB), we observe a noticeable share of Others (yellow). This is expected as small tile sizes increase the total number of loop overhead. On the other hand, although Line 7 dominates the theoretical I/O, its runtime

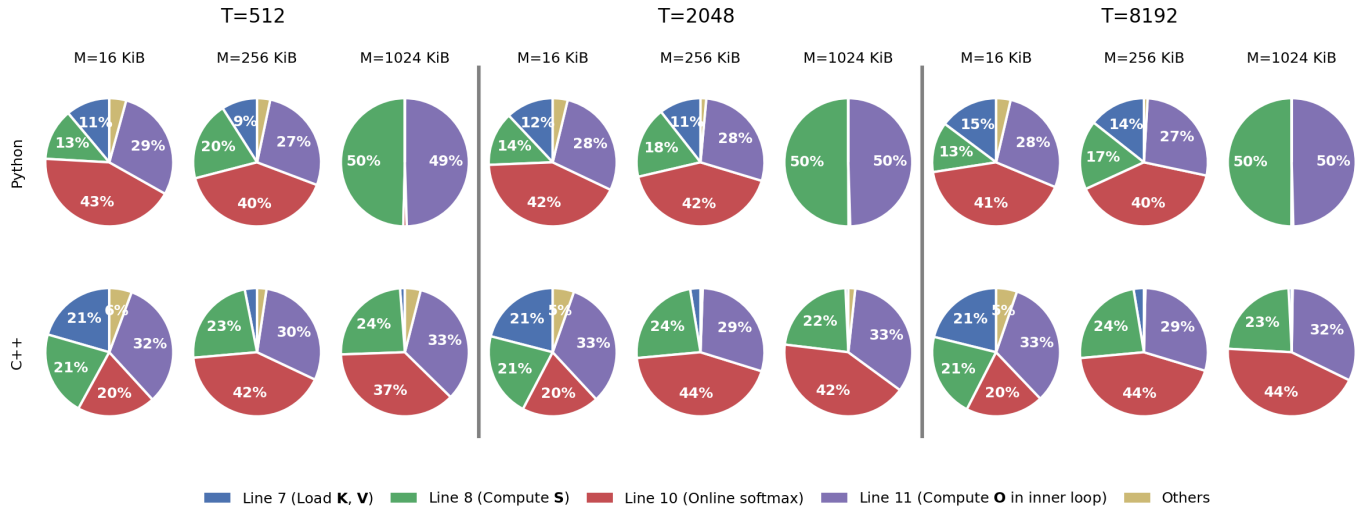

Fig. 4: Profiling results of major algorithmic steps of FlashAttention 2 (Algorithm 2) with Python (Numpy) and C++ (ST).

share remains relatively small and becomes negligible at $M = 1024$ KiB.

- Python Implementation: For $M \leq 256$ KiB, the costs are distributed among the four lines. However, at $M = 1024$ KiB, the profile collapses: Line 10 (online softmax) vanishes, and the matrix multiplications (Lines 8 and 11) saturate the runtime. This indicates that at large block sizes, NumPy matrix multiplications dominate the cost.
- C++ Implementation: In contrast, the C++ implementation maintains a stable distribution. Line 10 consistently consumes a significant portion of the runtime across all configurations.

Overall, our results demonstrate that a multi-threaded C++ implementation of FlashAttention-2 significantly accelerates long-context attention computations on CPU. Interestingly, although online softmax has a lower complexity, the empirical profiling shows it may be the bottleneck in certain configurations.

VII. Discussion

Our study has several limitations. First, our implementations prioritize algorithmic transparency over maximal hardware utilization. Without the hand-tuned SIMD vectorization and memory alignment standard in frameworks like llama.cpp, our results establish baseline algorithmic behavior rather than an upper bound for CPU inference. Second, we only test a single CPU architecture. Therefore, hardware-specific phenomena observed in our experiments may not generalize to different CPU architectures. Finally, we evaluate the forward pass kernel in isolation using synthetic inputs; end-to-end performance will depend on broader system factors like dynamic KV cache management and memory allocation overhead. Despite these limitations, this report serves as a critical preliminary exploration of the algorithm's behavior on CPU.

VIII. Conclusion

In this work, we evaluated the inference performance of the FlashAttention-2 forward pass on CPU and benchmarked single- and multi-threaded implementations against naive attention baselines. Our findings indicate that while a well-optimized naive C++ implementation serves as a highly competitive baseline for short sequences, parallelized FlashAttention-2 forward pass yields substantial performance gains for longer sequences. Empirical profiling reveals that the online softmax calculation can be a runtime bottleneck, despite having a smaller order of computational complexity compared to matrix multiplications. Our observations underscore the importance of further investigation of efficient attention algorithms on CPU.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

[2] T. Dao, "Flashattention-2: Faster attention with better parallelism and work partitioning," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=mZn2Xyh9Ec

[3] J. Jiang, J. Du, D. Huang, D. Li, J. Zheng, and Y. Lu, "Characterizing and optimizing transformer inference on arm many-core processor," in *Proceedings of the 51st International Conference on Parallel Processing*, ser. ICPP '22. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: https://doi.org/10.1145/3545008.3545022

[4] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," 2020. [Online]. Available: https://arxiv.org/abs/2004.05150

[5] B. Chen, T. Dao, E. Winsor, Z. Song, A. Rudra, and C. Ré, "Scatterbrain: Unifying sparse and low-rank attention," in *Advances in Neural Information Processing Systems*, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, Eds., 2021. [Online]. Available: https://openreview.net/forum?id=SehIKudiIo1

[6] K. M. Choromanski, V. Likhosherstov, D. Dohan, X. Song, A. Gane, T. Sarlos, P. Hawkins, J. Q. Davis, A. Mohiuddin, L. Kaiser, D. B. Belanger, L. J. Colwell, and A. Weller, "Rethinking attention with performers," in *International Conference on Learning Representations*, 2021. [Online]. Available: https://openreview.net/forum?id=Ua6zuk0WRH

[7] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret, "Transformers are RNNs: Fast autoregressive transformers with linear attention," in *Proceedings of the 37th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, H. D. III and A. Singh, Eds., vol. 119. PMLR, 13–18 Jul 2020, pp. 5156–5165. [Online]. Available: https://proceedings.mlr.press/v119/katharopoulos20a.html

[8] N. Kitaev, L. Kaiser, and A. Levskaya, "Reformer: The efficient transformer," in *International Conference on Learning Representations*, 2020. [Online]. Available: https://openreview.net/forum?id=rkgNKkHtvB

[9] A. Roy, M. Saffar, A. Vaswani, and D. Grangier, "Efficient content-based sparse attention with routing transformers," *Transactions of the Association for Computational Linguistics*, vol. 9, pp. 53–68, 2021. [Online]. Available: https://aclanthology.org/2021.tacl-1.4/

[10] M. Zaheer, G. Guruganesh, K. A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P. Pham, A. Ravula, Q. Wang, L. Yang, and A. Ahmed, "Big bird: Transformers for longer sequences," in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 17 283–17 297. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/c8512d142a2d849725f31a9a7a361ab9-Paper.pdf

[11] Y. Mao, M. Ester, and K. Li, "Iceformer: Accelerated inference with long-sequence transformers on CPUs," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=6RR3wU4mSZ

[12] T. Zhang, J. Yi, B. Yao, Z. Xu, and A. Shrivastava, "Nomad-attention: Efficient llm inference on cpus through multiply-add-free attention," in *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, Eds., vol. 37. Curran Associates, Inc., 2024, pp. 112 706–112 730. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/file/ccda3c632cc8590ee60ca5ba226a4c30-Paper-Conference.pdf

[13] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Re, "Flashattention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems*, A. H. Oh, A. Agarwal, D. Belgrave, and K. Cho, Eds., 2022. [Online]. Available: https://openreview.net/forum?id=H4DqfPSibmx